Hledac Universal — Final Deep Research Optimization Report for MacBook Air M1 8GB

Tato zpráva představuje naprosto vyčerpávající, na důkazech založenou architektonickou a výkonnostní analýzu projektu Hledač Universal. Cílem je poskytnout detailní náhled do optimalizačních strategií striktně navržených pro specifické hardwarové hranice zařízení MacBook Air M1 (2020) s 8 GB unifikované paměti. Vzhledem k vysoce asymetrické architektuře tohoto čipu – zahrnující čtyři výkonnostní (P) jádra, čtyři efektivní (E) jádra, šestnáctijádrový Neural Engine (ANE) a unifikovanou paměť sdílenou mezi procesorem a grafickým akcelerátorem – je nezbytné přistupovat k paralelizaci a správě paměti se zcela jinou filozofií, než jaká se uplatňuje u tradičních x86 serverových clusterů.[1]

Na tomto hardwaru vede agresivní paralelizace často k drastické destrukci výkonu vlivem zahlcení paměťové sběrnice (memory thrashing) a k vynucenému stránkování (swapování) operačního systému macOS, což má za následek neakceptovatelnou latenci autonomní rozhodovací smyčky.[1] Předkládaná analýza podrobně rozebírá přiložené auditní zprávy a zdrojové kódy, provádí jejich křížovou verifikaci a definuje, jakým způsobem maximalizovat propustnost a dlouhodobou stabilitu tohoto autonomního systému, aniž by došlo k narušení bezpečnostních bariér projektu.

1. Meta-summary of the attached reports

První fází hloubkové analýzy bylo kritické zhodnocení čtyř přiložených auditních dokumentů (HLEDAC_FINAL_CONSOLIDATED_OPTIMIZATION_REPORT.md, CLAUDE_CODE_DEEP_OPTIMIZATION_AUDIT_M1_8GB.md, EXTREME_OPTIMIZATION_SHORTLIST.md, EXTREME_OPTIMIZATION_AUDIT_M1_8GB.md). Tato syntéza odhaluje klíčové shody, identifikuje sporné body a upozorňuje na kritická architektonická slepá místa, která předchozí analýzy nedokázaly postihnout.

Při analýze konsensu mezi předchozími audity jasně vystupuje do popředí několik stěžejních problémů. Všechny reporty se shodují na existenci kritického úzkého hrdla v oblasti JSON serializace. Dokumentována je zejména dvojitá serializace v modulu evidence_log.py, kde volání funkce verify_integrity (která vnitřně obsahuje json.dumps) je spouštěno zbytečně dvakrát během jediné operace append.[1] Tato redundantní práce způsobuje masivní ztrátu procesorových cyklů. Dále panuje absolutní shoda na nutnosti odstranit inline kompilaci regulárních výrazů. Kompilace prováděná v rámci volaných funkcí (například v knowledge/graph_rag.py) zbytečně zatěžuje horké cesty (hot paths) a měla by být přesunuta na úroveň modulu.[1] Shoda panuje také v oblasti paměťové disciplíny pro čip M1; striktní politika "jeden model v paměti v jeden čas" (1-model-at-a-time) a zákaz agresivního načítání dat do

mezipaměti (prefetching) jsou vnímány jako naprosto nezbytné předpoklady pro zabránění vyčerpání paměti.[1] Významným bodem shody je rovněž identifikace chybějícího volání mx.eval() před příkazem mx.clear_cache() v MLX backendu, což vede k nechtěnému zadržování výpočetních grafů v paměti a postupnému úniku VRAM.[1]

Navzdory těmto shodám vykazují reporty významné rozpory v klíčových architektonických rozhodnutích. Značný rozpor panuje v otázce plošného nasazení MLX frameworku. Zatímco některé zprávy naznačují extenzivní využití MLX pro veškeré tenzorové výpočty na úkor knihovny NumPy, detailnější audity správně varují, že režie spojená s přesuny malých tenzorů mezi procesorem a grafickým jádrem (i přes existenci unifikované paměti) snadno převýší přínos samotné akcelerace u menších výpočtů.[1] Další zásadní neshoda se týká míry paralelizace vstupně-výstupních (I/O) operací. Jisté sekce auditů doporučují paralelizaci pomocí ThreadPoolExecutor, avšak jiné důrazně varují, že spuštění osmi vláken (což odpovídá hodnotě os.cpu_count()) na stroji s 8 GB RAM způsobí kolaps operační paměti.[1] V tomto sporu je nutné přiklonit se k druhému táboru: přísně ohraničená souběžnost (bounded concurrency) je na M1 8GB naprostou nutností.

Nejkritičtějším zjištěním této meta-analýzy je však to, co všechny přiložené reporty přehlédly. Přestože všechny dokumenty jednohlasně doporučují přechod ze standardní knihovny json na knihovnu orjson [1], zcela ignorují existenci modernější a paměťově výrazně efektivnější knihovny msgspec. Nezávislá referenční měření prokazují, že msgspec je na architektuře Apple Silicon přibližně 3krát až 4krát rychlejší při dekódování do struktur a 1,6krát rychlejší při kódování ve srovnání s Pydantic v2 a orjson.[4] Pro stroj omezený 8 GB paměti je menší paměťová stopa instancí msgspec.Struct (oproti běžným slovníkům dict nebo instancím pydantic) zcela zásadní, jelikož drasticky snižuje frekvenci spouštění garbage collectoru.[6]

Dále zprávy nezaznamenaly fatální chybu v souboru gnn_predictor.py, kde dochází k alokaci husté matice (dense matrix) prostřednictvím příkazu np.zeros((n_nodes, n_nodes)). Na systému s 8 GB paměti povede tento postup k havárii již při dosažení hranice přibližně 45 000 uzlů grafu (alokace překročí 8 GB fyzické paměti).[1] Řešením, které audity opominuly, je přechod na paměťově efektivnější grafové knihovny typu rustworkx, které zvládnou zpracovávat masivní grafy bez neúměrných nároků na operační paměť.[8]

Zcela opomenuta byla rovněž Apple-nativní vektorizace na úrovni procesoru. Namísto přetěžování GPU (přes MPS Graph) pro jednoduché výpočty kosinové podobnosti měly audity doporučit knihovnu SimSIMD. Tato knihovna přímo využívá instrukční sady AMX a NEON na čipech Apple Silicon a dosahuje při výpočtech úhlové vzdálenosti až 133násobného zrychlení ve srovnání s běžnou implementací v knihovně scipy.[10] Navíc reporty doporučující optimalizaci smyček pro extrakci metadat zcela ignorují možnost eliminovat paměťově náročné externí knihovny (jako PIL nebo ffmpeg) přímým napojením na nativní frameworky macOS (Vision, Natural Language, AVFoundation) prostřednictvím mostu PyObjC.[1]

Některá tvrzení v reportech byla navíc zveličena, například tvrzení, že plně asynchronní zápis

checkpointů na disk přinese masivní zvýšení propustnosti. Pokud operační systém macOS již na pozadí provádí silný I/O buffering, obalení tohoto zápisu do asynchronního vlákna v Pythonu přidává pouze režii event-loopu bez reálného zkrácení doby zápisu na SSD.[1] Naopak podceněna byla cena synchronních blokací uvnitř asynchronních cest. Použití importlib.import_module (lazy loading) uvnitř asynchronních bloků, jak bylo zjištěno v autonomous_orchestrator.py [1], představuje fatální hrozbu, neboť může na jednovláknové event loop zcela pozastavit činnost celého autonomního procesu.

2. Ground-truth findings from code inspection

Provedená hloubková kontrola poskytnutých zdrojových souborů (autonomous_orchestrator.py, hermes3_engine.py, gnn_predictor.py, metadata_extractor.py) a jejich konfrontace s tvrzeními z auditů umožňuje rozlišit skutečný stav architektury od teoretických předpokladů.

To, co je prokazatelně reálné, ukazuje silné povědomí původních vývojářů o hardwarových limitech. Striktní kontrola souběžnosti v jádře modelového enginu je ukázkově implementována. Soubor hermes3_engine.py inicializuje omezovač ThreadPoolExecutor(max_workers=1) (file:line 135) v kombinaci se synchronizačním primitivem asyncio.Semaphore(1) (file:line 137). Toto architektonické rozhodnutí bezchybně zabraňuje kolizi více modelů v paměti a naplňuje tak guardrail politiku M1 8GB.[1] Stejně prokazatelná je implementace kontinuálního dávkování (continuous batching), kde asynchronní worker na pozadí efektivně akumuluje požadavky s maximální velikostí dávky 8 a krátkým timeoutem 50 ms (lines 142-143, 177-208), čímž optimalizuje propustnost modelu, aniž by došlo k zahlcení paměti.[1] Velmi inovativním a funkčním prvkem jsou dynamické paměťové zábrany (memory guards) pro spekulativní dekódování (speculative decoding). Logika inteligentně vybírá draft model na základě aktuálně dostupné RAM: při dostupnosti nad 5,5 GB zavádí 1B model, nad 4,0 GB 100M model a při kritickém nedostatku paměti tuto funkci zcela deaktivuje (lines 246-259).[1]

Na druhou stranu inspekce potvrdila i přítomnost kritických chyb. Jak již bylo naznačeno, alokace dense matice v gnn_predictor.py příkazem np.zeros((n_nodes, n_nodes), dtype=np.float32) skutečně v kódu figuruje a představuje tikající časovanou bombu pro stabilitu systému.[1] Nalezena byla rovněž prokazatelná přítomnost synchronních blokací v srdci asynchronního orchestrátoru. Soubor autonomous_orchestrator.py obsahuje metodu AnalysisSchema.from_dict(cls, data: Dict), která při častém volání nad velkými datovými sadami vytváří nebezpečný procesorový hotspot.[1] Podobně problematické je synchronní volání knihovny ctypes a rozhraní sysctlbyname pro ověření dostupnosti Metal limitů, které nepřípustně blokuje asynchronní event loop.[1]

V rovině silně pravděpodobných zjištění (označeno jako PROBABLE, vyžadující dodatečné provozní měření) se nachází jev, který lze nazvat "memory thrashing při extrakci metadat". Modul metadata_extractor.py proklamuje využití lazy loadingu pro importy knihoven PIL, pypdf, docx a ffmpeg přímo uvnitř extrakčních metod.[1] Ačkoliv se jedná o dobrou praxi pro rychlý start

aplikace, v dlouhodobém autonomním běhu, kdy orchestrátor začne v těsném sledu analyzovat heterogenní soubory, dojde k souběžnému zavedení všech těchto těžkých binárních rozšíření do paměťového prostoru. To vyvolá skokový nárůst spotřeby RAM hluboko nad uvažovaný bezpečnostní limit 500 MB na extrakci, což na 8GB stroji okamžitě vyvolá stránkování a destrukci cache uložených tenzorů LLM modelu. Stejně pravděpodobné jsou postupné úniky paměti (memory leaks) v grafové predikci. V gnn_predictor.py neustále roste atribut self.node_features = {} bez implementace jakékoliv vyklízecí (purging) logiky nebo omezení typu LRU. U dlouhých autonomních runů tento slovník s trvalou referencí postupně zkonzumuje veškerou volnou RAM.[1]

A konečně, některá tvrzení z auditů zůstala během inspekce zcela neověřená (UNVERIFIED). Hlavním bodem je tvrzení o dvojím volání verify_integrity() v modulu evidence_log.py, které způsobuje redundanci v generování JSONu a hashů.[1] Jelikož samotný zdrojový soubor evidence_log.py nebyl k inspekci fyzicky poskytnut (k dispozici byla pouze reference na něj), nelze toto s jistotou potvrdit zkoumáním zdrojového kódu. Avšak analýza navazujících komponent, jako je BudgetManager v autonomous_orchestrator.py sledující metriku max_snapshot_writes, vysoké procesorové režii tohoto mechanismu silně nasvědčuje.[1] Ověření bude vyžadovat detailní profilování exekučních časů uvnitř komponenty EvidenceLog.

3. Deep Candidate Analysis

Tato sekce obsahuje do absolutní hloubky propracovanou analýzu sedmi nejvýznamnějších optimalizačních kandidátů s přihlédnutím ke všem myslitelným vlivům na architekturu M1.

Candidate 1: Global JSON Serialization Hotpaths

1. Location:

- autonomous_orchestrator.py (řádek 38 import json), hypoteticky evidence_log.py a knowledge/atomic_storage.py (odkazy z auditů).
- Funkce: AnalysisSchema.from_dict a serializace v AtomicJSONKnowledgeGraph.[1]

2. Current behavior:

Systém aktuálně používá vestavěnou knihovnu json pro ukládání stavových checkpointů, serializaci znalostního grafu a transformaci LLM výstupů přes slovníky. Toto je kritická horká cesta (hot path) volaná potenciálně tisíckrát za minutu.

3. Why it is expensive or suspicious: Standardní json.dumps a json.loads jsou extrémně pomalé. Vytvářejí v Pythonu rozsáhlé stromové struktury objektů (typicky dict a list), které vedou k agresivní fragmentaci haldy (heap). O(n log n) komplexita při povolení sort_keys=True tento problém dále zhoršuje.[1] Tento masivní object churn ničí cache lokalitu čipu M1 a spouští neustálé pauzy garbage collectoru.

4. Best recommendation:

REPLACE WITH MORE MODERN METHOD

5. Best hardware target:

P-cores (výhradně). Úloha je čistě limitována taktem CPU a rychlostí alokace paměti; offloading na GPU zde nemá smysl.

6. Why this is the best fit for MacBook Air M1 8GB:

Paměťová topologie M1 spoléhá na extrémně rychlé vyrovnávací paměti (L1/L2) sdílené napříč P-jádry. Menší paměťová stopa a kontiguální rozložení dat v paměti vede k dramatickému nárůstu výkonu. Minimalizace alokací na haldě zajišťuje, že M1 nebude nucen žádat o paměťové stránky z fyzické UMA paměti.

7. Better modern / cutting-edge alternative:

Navzdory tomu, že předchozí audity navrhují přechod na orjson, reálnou špičkovou alternativou je knihovna msgspec.

- *Proč je skutečně lepší:* msgspec umožňuje dekódovat JSON přímo do msgspec.Struct typů definovaných v Pythonu. Tyto struktury jsou paměťově mnohonásobně menší než tradiční Python slovníky, alokují data lineárně podobně jako C-struktury a provádějí validaci schématu během jednoho průchodu pamětí (zero-cost schema validation).[6] Benchmarky prokazují, že dekódování do struktur je až 6krát rychlejší než Pydantic a kódování je srovnatelné nebo lepší než orjson.[4]
- *Proč by to mohl být špatný nápad:* msgspec je striktní, vrací objekty typu bytes namísto str a nepodporuje bez úprav všechny "magické" funkce frameworků třetích stran (není plným drop-in replacementem pro Pydantic ve všech komplexních vnořeních).[14] Realistické je to okamžitě pro izolované vrstvy.

8. Risk analysis:

- Regression risk: Střední (změna typování z str na bytes při dumpingu).
- Complexity risk: Nízký (deklarace schémat je intuitivní).
- Observability need: Nízká.
- Correctness risk: Velmi nízký (striktní typová kontrola zabraňuje vzniku nevalidních stavů).

9. Validation plan:

Vytvoření deterministického benchmarku: 1000 iterací serializace a deserializace databáze AtomicJSONKnowledgeGraph. Snížení časové náročnosti musí být minimálně 75 % a sledování alokace paměti přes moduly tracemalloc musí ukázat pokles paměťového overheadu alespoň o 30 %.

10. Classification:

SAFE QUICK WIN pro ohraničené části (logy, jednoduché checkpointy). HIGH-RISK ARCHITECTURAL CHANGE pro úplnou náhradu všech interních datových modelů orchestrátoru.

Candidate 2: Dense Adjacency Matrix w/ $O(N^2)$ Growth

1. Location:

- gnn_predictor.py
- Třída GNNPredictor, metoda predict a funkce train_gnn_task (řádky cca alokace adj_np).[1]

2. Current behavior:

Grafová neuronová síť naivní metodou generuje hustou matici sousednosti: adj_np = np.zeros((n_nodes, n_nodes), dtype=np.float32). Tuto matici posílá do modelu k výpočtu adj @ x.

3. Why it is expensive or suspicious: Z matematického hlediska roste paměťová složitost této operace kvadraticky ($O(N^2)$). Každá položka typu float32 vyžaduje 4 bajty. Graf se 45 000 uzly (což u OSINT grafů vzniká po pár hodinách chodu) se pokusí alokovat 8,1 GB jednolité blokové paměti.[1] Toto chování spolehlivě způsobí zhroucení aplikace nebo neovladatelné SSD swapování trvající minuty.

4. Best recommendation:

REPLACE WITH MORE MODERN METHOD

5. Best hardware target:

CPU mixed.

6. Why this is the best fit for MacBook Air M1 8GB:

Fyzický limit 8 GB paměti je nepřekonatelný. Protože přes GPU běží MLX inference (obsazující cca 3–4 GB), aplikace nemůže pro pomocné datové struktury využívat více než 1–2 GB RAM, aniž by došlo ke kolapsu. Zde jde primárně o záchranu systému před selháním (OOM).

7. Better modern / cutting-edge alternative:

Nahradit naivní dense matice v NumPy za struktury založené na Seznamech Hran (Edge List / Edge Index) nebo využít vysoce optimalizovanou knihovnu rustworkx.

- *Proč je lepší:* rustworkx reprezentuje grafy efektivně pod kapotou v jazyce Rust a vyhýbá se plnému rozbalování do matic. Pro výpočet samotné sítě se musí upravit forward pass z čisté maticové multiplikace na takzvané scatter/gather agregace po sousedech.[8] Namísto

8 GB paměti zabere graf se 45 000 uzly paměť v řádu nižších desítek megabajtů.
- *Realističnost:* Naprosto nutná architektonická úprava, pokud má systém operovat dlouhodobě.

8. Risk analysis:

- Regression risk: Značný, jelikož se musí přepsat vstupní vrstva topologie GraphSAGE.
- Correctness risk: Musí se pečlivě verifikovat, že násobení přes indexy hran matematicky odpovídá hustému násobení adj @ x.

9. Validation plan:

Testování alokace: Spuštění mockovaného tréninkového cyklu pro 50 000 uzlů a měření RSS (Resident Set Size). Test projde, pokud využití operační paměti zůstane po celou dobu trvání testu stabilně pod hranicí 100 MB.

10. Classification:

HIGH-RISK ARCHITECTURAL CHANGE (avšak s imperativem absolutní nevyhnutelnosti).

Candidate 3: Vector Similarity Operations

1. Location:

- Předpokládané lokace: knowledge/lancedb_store.py (dle reportů), dále gnn_predictor.py nebo scoring utils.

2. Current behavior: Systém využívá standardní operace z balíků numpy nebo scipy.spatial.distance.cosine pro vyhledávání sémantické podobnosti nad vygenerovanými 1536-dimenzionálními embeddingy (např. z modelu Ada nebo lokálního embedderu).[1]

3. Why it is expensive or suspicious: Výpočty v numpy používají tradiční úroveň implementace nad frameworky BLAS/LAPACK. Ačkoliv jsou tyto knihovny rychlé pro makro-matice, pro časté porovnávání menších vektorových párů vytvářejí ohromný režijní čas v interpretační smyčce Pythonu. Každé spuštění trvá příliš dlouho na to, aby mohlo probíhat v objemech reálného času v rámci RAG cyklů.[10]

4. Best recommendation:

APPLE-NATIVE ACCELERATION

5. Best hardware target:

P-cores využívající instrukce AMX (Apple Matrix Coprocessor) a NEON SIMD.

6. Why this is the best fit for MacBook Air M1 8GB:

Urychlení úzce zaměřených matematických operací na procesoru znamená, že grafický akcelerátor GPU zůstane plně a dedikovaně vyhrazen pro generování textu přes framework MLX. Architektura M1 disponuje instrukcemi NEON, které zvládají zpracovávat masivní vektory zcela deterministicky a bez jakékoliv spotřeby dodatečné operační paměti.

7. Better modern / cutting-edge alternative:

Knihovna SimSIMD.

- *Proč je lepší:* Tato knihovna je ručně psaná v jazyce symbolických instrukcí a C přímo pro architekturu ARM NEON/SVE. Při měření rozdílů ve výkonu oproti standardní knihovně scipy poskytuje na M1 čipech až 133násobné zrychlení při výpočtech kosinové podobnosti u 1536D vektorů, a to bez potřeby bootování masivních instancí.[10]
- *Proč by to mohlo být nevhodné:* Jde o externí binární závislost, nicméně knihovna poskytuje předpřipravené pip balíčky přímo pro platformu macosx-arm64.[16]

8. Risk analysis:

- Complexity risk: Minimální. Rozhraní SimSIMD slouží jako absolutní přímá náhrada (drop-in replacement).
- Correctness risk: Prakticky nulový, využívají se vysoce testované matematické instrukce.

9. Validation plan:

Mikro-benchmark porovnávající starou a novou metodu nad množinou 10 000 embeddingů. Doba provádění musí klesnout o >95 %, výsledné hodnoty podobnosti (float) se nesmí lišit o více než toleranci ε=1e-5.

10. Classification:

SAFE QUICK WIN.

Candidate 4: Asynchronous Lazy Loading Blockage

1. Location:

- autonomous_orchestrator.py, sekce implementace třídy _LazyModule a jejího použití (zhruba řádky kolem bloků inicializací).[1]

2. Current behavior:

Za účelem úspory operační paměti dochází k oddálenému zavedení velkých frameworků (jako je torch nebo mlx_lm) pomocí importlib.import_module(). K tomuto zavádění dochází dynamicky ve chvíli prvního přístupu k příslušným atributům, a to přímo v těle asynchronních procedur.

3. Why it is expensive or suspicious:

Funkce z rodiny importlib představují blokující, synchronní operace operačního systému (OS), které zahrnují stovky přístupů na souborový systém a kompilaci modulů z jazyka C. Spuštění takto drahé operace uvnitř metody definované jako async na jednovláknové smyčce asyncio okamžitě způsobí kompletní zastavení procesování (tzv. "World Stop") všech ostatních událostí a naplánovaných úloh, někdy i na několik vteřin. To je v reálném čase reagujícím softwaru nepřípustné.

4. Best recommendation:

THREAD PARALLELIZE

5. Best hardware target:

E-cores (Efficiency cores) pro provádění operací zaměřených na práci se souborovým systémem na pozadí.

6. Why this is the best fit for MacBook Air M1 8GB:

Architektura 4P+4E umožňuje vyčlenit nízkonákladové blokující V/V (I/O) úlohy na efektivní jádra. Hlavní P-jádro dedikované pro logiku asyncio event loopu pak není těmito operacemi brzděno a udržuje si maximální odezvu pro orchestraci. Z hlediska celkové spotřeby paměti je zachována logika zpožděného nahrávání, ale odstraňuje se destruktivní blokace.

7. Better modern / cutting-edge alternative:

Obalení logiky pozdního nahrávání (lazy loading) pomocí nástrojů asyncio.to_thread v kombinaci s budováním pre-warm (zahřívacích) rutin v inicializační fázi orchestrátoru. Těžké moduly se začnou načítat ve speciálním vlákně na pozadí, hned jak začne klidová fáze systému.

8. Risk analysis:

- Regression risk: Vysoký. Při souběžném přístupu k dosud nekompletně zavedenému modulu může vzniknout souběh procesů (race condition).
- Guardrail compatibility: Mimořádně důležité. Změna nesmí narušit existující restrikce na 8GB M1 (moduly se stále nesmí nahrávat všechny naráz do paměti při spuštění programu). Vyžaduje to použití mechanismu asyncio.Lock kolem samotného procesu importu.

9. Validation plan:

Aktivace sledování zpoždění event loopu (asyncio.get_event_loop().slow_callback_duration). Žádný tick (obrátka smyčky) nesmí trvat déle než 100 milisekund během prvního zavolání MLX modelů.

10. Classification:

MEASUREMENT-FIRST.

Candidate 5: "Heavy" Media Metadata Extraction Frameworks

1. Location:

- metadata_extractor.py.[1]

2. Current behavior:

Systém aktuálně provádí skenování a těžbu metadat z heterogenních souborů tím způsobem, že "lenivě" importuje nezávislé knihovny třetích stran, konkrétně PIL pro obrázky, mutagen pro audio, pypdf pro dokumenty a využívá podprocesy spouštějící nástroj ffmpeg pro zpracování video obsahu.

3. Why it is expensive or suspicious:

Tyto knihovny spotřebovávají masivní množství paměti v prostoru samotného Pythonu, což nutí Python instanci neustále žádat OS o nové alokace. Spouštění procesu ffmpeg vytváří klon paměťového kontextu a zahlcuje výpočetní kanály (Context Switch). Naráz se provádějící dávkové zpracování několika souborů tyto instance nastřádá v paměti a s jistotou prolomí definovaný bezpečnostní limit "500 MB per extraction", čímž odsoudí celý systém M1 8GB ke spuštění swapovacího mechanismu, což zásadně prodlužuje aplikační latence.

4. Best recommendation:

APPLE-NATIVE ACCELERATION

5. Best hardware target:

Hardware-akcelerovaný Media Engine čipu M1 a Unified Memory výhoda.

6. Why this is the best fit for MacBook Air M1 8GB: Aplikace, které přímo využívají nativní subsystémy operačního systému macOS (např. přes AVFoundation pro videa nebo ImageIO pro fotky), pracují s architekturou "zero-copy". Rozhraní operačního systému načte stream dat přímo do hardwarových dekódovacích jednotek, extrahuje metadata v řádu milisekund a uvolní zdroje, aniž by tyto gigabajty dat vůbec zasáhly aplikační haldu (heap) Python interpreteru.[1]

7. Better modern / cutting-edge alternative:

Vytvoření úzce zaměřených překladačů pomocí modulu PyObjC, které napřímo komunikují s knihovnami systému macOS:

- Obrazy: Přechod z PIL na Vision framework a CoreGraphics / ImageIO.
- Videa a zvuk: Přechod z ffmpeg a mutagen na instanci AVAsset poskytovanou

frameworkem AVFoundation.
- PDF Dokumenty: Přechod z pypdf na PDFKit (využívá nízkoúrovňový Quartz).
- *Realističnost:* Velmi realistické pro dedikované self-hosted nasazení výhradně pro stroje Apple, vyžaduje se však jednorázové studium principů a metod v PyObjC.

8. Risk analysis:

- Complexity risk: Vysoký. Vývoj v PyObjC a nutnost provádět ruční správu alokačních bazénů (Autorelease Pools) v multithreadovém Python běhu je náchylné na chyby implementace.
- Platform lock-in: Absolutní. Systém Hledač pak nepoběží na jiném OS (což, podle hard constraints tohoto auditu, není problém).

9. Validation plan:

Spuštění paralelizované extrakce 1 GB video souboru a 100 MB obrazového materiálu. Paměťový stopový graf procesu Pythonu by se neměl zvednout o více než 50 MB, neboť dekódování zajistí nízkoúrovňový démon macOS.

10. Classification:

APPLE-SILICON HIGH-UPSIDE EXPERIMENT.

Candidate 6: LLM Speculative Decoding and Draft Model Thrash

1. Location:

- hermes3_engine.py, funkce _init_draft_model (řádky cca 236–267).[1]

2. Current behavior: Motor pro běh rozsáhlých jazykových modelů se snaží optimalizovat rychlost generování tokenů použitím techniky spekulativního dekódování (speculative decoding). Pokud má systém dostatek volné RAM (detekováno např. >5,5 GB), zavedou se do paměti simultánně dva modely: rozsáhlý primární cíl (typicky 3B parametrů) a menší generátor návrhů (draft model o velikosti 100M až 1B parametrů).[1]

3. Why it is expensive or suspicious:

Pro udržení dvou jazykových modelů současně (včetně jejich výpočetních vah a oddělených vyrovnávacích pamětí KV Cache) ve sdíleném 8GB poolu architektury M1 vyvstává mimořádně závažný problém s propustností. Modely o paměť soupeří, což u čipu s propustností sběrnice přibližně 68 GB/s tvoří limitující hrdlo (memory bandwidth constraint).

4. Best recommendation:

MICRO-OPTIMIZE (při striktním vynucení paměťových technik).

5. Best hardware target:

GPU (správa paměti přes MLX Metal engine).

6. Why this is the best fit for MacBook Air M1 8GB:

Jakákoliv neopatrnost u M1 8GB povede k vyčerpání paměti pro vrstvy pozornosti (attention layers) a pádu generace. Primární model v konfiguraci 4-bit (cca 2 GB) a podpůrný model (cca 0,7 GB) samy o sobě zaberou jen malou část prostoru, nicméně narůstající vyrovnávací paměť KV Cache po vložení tisíců tokenů kontextu zhltne klidně další 2 až 3 GB. S tím je systém postaven před neodvratnou havárii.

7. Better modern / cutting-edge alternative: Naprostým a neměnným technologickým předpokladem pro smysluplné zachování spekulativního dekódování na cílovém zařízení je zapnutí agresivní, plošné 4-bitové kvantizace samotné vyrovnávací paměti KV Cache, tedy takzvané techniky kv4 pro MLX.[17] Ve zdrojovém kódu existuje logika na její zjištění (řádky 292-295 _init_system_prompt_cache()), avšak musí být hardcoded pro obě varianty modelu.[1] Příklad: pro běžný kontext klesne u 4bitové struktury spotřeba RAM z obvyklých 1,60 GB na nepatrných 0,51 GB.[17] Alternativou na prahu špičkového výzkumu je zahození návrhového modelu (draft) a adopce moderních architektur se zabudovanou podporou předpovědi více tokenů zároveň (MTP - Multi-Token Prediction formát, například eagling), kde veškerá práce zůstává uvnitř vah jednoho síťového jádra, a VRAM se neštěpí.[18]

8. Risk analysis:

- Degradation risk: Redukce KV Cache na 4 bity (kv4) s sebou nese malou, přesto měřitelnou degradaci schopnosti modelu reflektovat extrémně dlouhé pasáže textu z úvodu kontextového okna (tzv. "ztráta pozornosti", attention fading). Záleží, nakolik je Hledač citlivý na RAG halucinace.

9. Validation plan:

Srovnávací test spotřeby. Inicializace 4K tokenů RAG kontextu a generace 500 tokenů. Monitorování aktivity sběrnice pomocí integrace nástroje powermetrics k odhalení saturace RAM šířky pásma a logování mx.metal.get_active_memory().

10. Classification:

MEASUREMENT-FIRST.

Candidate 7: Unbounded Storage Structures

1. Location:

- Znalostní graf: Třída GNNPredictor, objekt self.node_features = {}.[1]
- Další interní struktury: Hypoteticky různé seznamy a logovací buffer paměti, které rostou s

počtem proběhlých cyklů.

2. Current behavior:

Objekt mapující uzly na embeddingy (node_features) je inicializován jako prostý standardní slovník v Pythonu a s každým novým zpracovaným zdrojem nekonečně bobtná. Neobsahuje žádnou politiku stárnutí, omezování (pruning) či ukládání na disk.

3. Why it is expensive or suspicious: Je zřejmé, že se jedná o nebezpečný memory leak v architektuře dlouho běžícího systému. Jenom samotných 10 000 embeddingů v rozlišení float32 pro 1536 dimenzí spolkne okamžitě kolem 60 megabajtů hrubé kapacity. Ve chvíli, kdy graf naroste na miliony uzlů, zhroutí se Python prostředí s klasickou chybovou hláškou OS "Out of Application Memory", a to zcela nezvratně a fatálně.[10]

4. Best recommendation:

BATCH + CACHE (Služby s LRU stárnutím).

5. Best hardware target:

Unified Memory, sdílené prostředky pro stabilní long-run stav.

6. Why this is the best fit for MacBook Air M1 8GB:

Aplikace "Bounded-as-airbag" guardrailu v praxi. Na 8GB stroji je stabilita operační paměti nadřazena i propustnosti. Každá vnitřní struktura musí mít svůj strop, jinak systém nevyhnutelně spadne.

7. Better modern / cutting-edge alternative:

Nahrazení slovníku za ohraničenou datovou strukturu (například wrapper nad functools.lru_cache, collections.OrderedDict v režimu FIFO, či dedikovaná implementace bounded map). Nebo mnohem lépe, vzhledem k přítomnosti LanceDB (knowledge/lancedb_store.py), vytlačovat embedding uzlů plně do vrstvy MMap storage formátu LanceDB sídlícího na rychlém M1 SSD a v RAM držet pouze stovky nejaktivněji využívaných uzlových znaků (Features) napojených na aktivní operace GNN modelu.

8. Risk analysis:

- Complexity risk: Nízká úroveň. Jde pouze o úpravu přístupových getterů.
- Correctness risk: Musí se zajistit ošetření scénáře "Cache Miss" v okamžiku, kdy GNN prediktor vyžaduje z RAM data, která již byla vystrnaděna do SSD úložiště.

9. Validation plan:

Dlouhodobý stresový zátěžový test: vložení půl milionu fingovaných (mock) uzlů do struktury

spojené se sledováním RAM. Pokud nedojde k překročení tvrdě fixované bariéry konzumace, test dopadl úspěšně.

10. Classification:

SAFE QUICK WIN.

---

4. Micro-Optimizations Ledger

Tyto méně robustní, nicméně kumulativně velmi podstatné optimalizace drobných nedostatků se skládají do efektivnějšího celku a perfektně ladí s pravidlem "Prefer small hot-path simplifications" definovaným mantinely projektu:

Optimalizační vektor	Aktuální stav / Problém v kódu	Mikro-zásah pro M1 8GB
Hashing Overhead	Rozsáhlé využívání modulu hashlib.sha256 na mnoha místech pro procesy nesouvisející s kryptografií (deduplikace řetězců, mapování, logy).[1]	Okamžitá a plošná náhrada volání hashlib.sha256 rychlou knihovnou xxhash, která je pro prokazatelně ne-kryptografické účely na ARM platformách o 1000 % rychlejší (až 10x).[20]
Regex Compilation	V hot-path funkcích kód dynamicky volá funkce re.search() nebo re.findall(), což uvnitř těla kompiluje state-machine vzorů opakovaně znova a znova.[1]	Refaktorovat moduly graph_rag.py a analytiku (normalize_text, has_contradiction). Explicitně deklarovat vykompilované objekty do globálního modulu pomocí syntaxe _PATTERN = re.compile(). Zrychlí běh funkce bez alokačního růstu.
Logging Guardrails	Obrovské množství 3255 zaznamenaných ladících logování (logging calls), jako	I vypnutý logovací level nezabrání spuštění pomalé metody json.dumps() nebo

	jsou hluboké výpisy stavů přes interpolace f"State: {json.dumps(obj)}".[1]	formátování stringů. Okamžité zaobalení drahých interpolací a trasovacích zpráv do hlídače: if logger.isEnabledFor(logging .DEBUG): logger.debug(...). Uspoří zbytečné O(N) výpočty.[1]
Garbage Collection	Záznamy o paměťových únicích v MLX bězích.[1] MLX Metal API zadržuje cache dokud se nevynutí materiální výsledek na úrovni Python haldy.	Před každým hovorem na uvolnění mx.metal.clear_cache() nutně zakotvit a evaluovat rozpracované stromy paměti zavoláním mx.eval(). Zabrání se tak vzniku fantomových referencí a vylepší se čistota Metal Heap poolu, což M1 zamezí pádům na nedostatku OOM.
Payload Object Churn	Manipulace s textem systémové nápovědy u modelu v souboru hermes3_engine.py (tzv. system_prompt).	System prompt by měl být zpracován, předpočítán a fixně serializován v novém msgspec.Struct balíčku pouze jednou v prefill startu a načten z disku rovnou v binární paměťové formě .npz. Snižuje tak počet alokací nových řetězců při každém požadavku.[1]
Math Scoring Ops	V případě nasazení numerického vyhodnocování (O(1) logiky v Action Scorerech) hrozí pokušení přesunout výpočet na GPU/MLX.	Neoptimalizovat tyto skórovací funkce do GPU.[1] Matematika přes dict lookupy nebo primitivní násobení na P-jádrech je drasticky efektivnější a levnější, než vytváření synchronizačních zámků

		pro frontu MLX grafiky M1.

---

5. Parallelization / Concurrency Map

Na stroji typu MacBook Air M1 s 8 GB UMA neexistuje paralelizace "zadarmo". Jakékoliv otevřené přidávání vláken v asynchronním I/O světě nevyhnutelně směřuje k zahlcení fyzických hranic systému. Toto je architektonická mapa, kde paralelizace poslouží a kde naopak usmrtí projekt:

Co bezpečně paralelizovat a izolovat (The "Worthwhile" Concurrency):

- **Disk I/O a Checkpointing:** Zápis dat do evidence_log a asynchronní generování snapshotů stavu znalostního grafu (přes LMDB) lze, resp. dokonce je nutné, vytěsnit mimo hlavní event loop. Musí být zabaleny do kontextu asyncio.to_thread v rámci chráněného fondu vláken.
  - *Hardware fit & Limits:* Striktně limitovat přes instanci ThreadPoolExecutor(max_workers=2). Přenese se tím zátěž logování na vysoce efektivní E-cores procesoru (která spotřebovávají nepatrně Wattů) a zanechají P-cores volné pro bystré rozhodování AI fronty.[1]
- **Sběrač a Fetch Coordinator (Network Requests):** Stahování souborů a parsování.
  - *Hardware fit & Limits:* Nutno aplikovat asyncio.Semaphore(3). Paralelizace sběru nad úroveň 3 požadavků ve stejnou vteřinu jednak neprospívá proxy/maskování sítě, a především zahltí mezipaměť (buffers) TCP soketů. Rovněž spustí najednou nadbytečné instance modulu selectolax, což ohrožuje UMA RAM.

Co je striktně ZAKÁZÁNO paralelizovat (The "Do NOTs" of 8GB Architecture):

- **MLX Model Inference (hermes3_engine.py):** Předchozí audity zmiňují, že model využívá semaphore s propustností 1. To je fantastická ochrana, která musí být zachována za každou cenu.[1] Pokusy nechat běžet inferenci velkého jazykového modelu paralelně vedle inferencí modelů klasifikačních generují tzv. "Context Swap" stav a pád na OOM, v lepším případě způsobí zablokování grafického ovladače Metal API a sníží počet vygenerovaných tokenů za sekundu (t/s) o 90 %.
- **Hutné zpracování CPU textů (CPU-Heavy JSON/HTML parsování):** Třebaže se nabízí uvolnit smyčku a nahrát serializační JSON převody AnalysisSchema.from_dict do samostatných procesních workerů, je to zásadní omyl. Staré zpracování dat uvolňuje takzvaný GIL jen s obtížemi a náklady na IPC (komunikaci mezi procesy a kopie paměti ve sdílené sběrnici) sráží M1 CPU dolů.[1] *Řešení:* Přejít na C-akcelerovanou strukturu msgspec a ponechat v sekvenčním vláknu hlavní cesty bez dodatečné paralelizace.
- **Hodnotící funkce (Graph Traversal & Scoring):** Jakékoli úvahy o souběžném počítání vah mezi kontextovými mapami O(1) by přinesly do kódu zámky na uzly a složitou režii souběžnosti. Jednoduchý for-cyklus zpracovaný v L1 cache výkonnostního jádra

procesoru je nedostižně efektivnější.[1]

Závěr kapacity: Hardwarová rovnováha RAG OSINT Agenta na cílovém stroji musí znít takto: Ne více než 2 aktivní vlákna pro I/O na pozadí, výhradně 1 dominantní asynchronní smyčka na logiku a přesně 1 souvislý proces uzamčený pro GPU MLX tenzory.

---

6. Apple Silicon Opportunity Map

Přístroj obsahuje hardwarové akcelerátory optimalizované pro konkrétní instrukční matematiku a multimédia. Tato část ukazuje, které nativní příležitosti je realistické na architektuře využít.

Apple Neural Engine (ANE) / CoreML Candidates:

Na čipu dřímá ohromný potenciál v 16jádrovém Neural Engine, který umí poskytnout výkon ~11 bilionů operací za sekundu při spotřebě zlomku wattu, ovšem nedosahuje dynamické flexibilnosti GPU.

- *Příležitost:* Odstřihnout model zpracovávající generování textových embeddingů (např. nízkoúrovňový rag_engine kódování vět) z backendu MLX a plně ho zkompilovat a exportovat pro vrstvu CoreML s vynuceným spouštěním pouze na **ANE**.[21]
- *Zisky na 8GB M1:* Zásadní uvolnění grafického subsystému a jeho dedikované RAM paměti výlučně pro textový LLM, čímž vzniká pravý překrývající se stav nulové spotřeby a vyšší celkové propustnosti za minutu.

Natural Language Framework Candidates:

Nativní aplikační rozhraní macOS.

- *Příležitost:* Pokud systém využívá složité vyhodnocování sentimentu, detekci jazyka nebo prvotní identifikaci entit (NER) přes obrovské importované knihovny jako Transformers nebo spaCy, jsou ideálním adeptem na destrukci. Tyto modely lze na operačním systému nahradit přímým voláním modulu NaturalLanguage pomocí frameworku PyObjC, který pro analýzu textu využívá zabudované váhy operačního systému sdílené celým macOS.[1]
- *Zisky:* Ohromná redukce paměti Python interpretu.

Accelerate / vDSP / SimSIMD Candidates:

Namísto standardního, unifikovaného přístupu softwarové implementace dot-productů je možné na hardwarové úrovni oslovit speciální registr instrukcí.

- *Příležitost:* Nahradit všechny moduly pro výpočet vzdáleností sémantických vektorů (cosine similarity) a bodového součinu využitím frameworků Apple Accelerate a zejména jeho adaptace přes projekt SimSIMD.[10] Ten aktivuje vektorizační AMX a NEON moduly rovnou do M1 P-jader. Rychlost se pohybuje u RAG porovnávání řádově v naprostých špičkách bez ztráty přesnosti nebo nárůstu tlaku na teplo stroje.[23]

Which to REJECT and why (Metal MPS Graph, PyTorch MPS):

- Je naprosto mylné předpokládat, že pokud převedeme libovolný NumPy drobný aritmetický úkol na akceleraci prostřednictvím PyTorch MPS (Metal Performance Shaders), získáme vyšší rychlost.
- *Důvod zamítnutí pro M1:* Přesunutí dat malého tenzoru (jako je pár malých matic o tisíci buňkách) ze subsystému P-jádra CPU skrze řadič UMA do oblasti paměti přiřazené grafickému koprocesoru (a čekání na spuštění kernelu na 8jádrovém GPU) trvá o celé řády déle, než kdyby tyto drobné vektory sečetla jednoduchá vektorová CPU instrukce. Implementace Metal/MPS backendu na dílčí logiku by zároveň vytvářela soupeření o grafické prostředky a neustálé narušování MLX vrstvy běžící LLM.[1] Experiment s nahrazením lehkého NumPy za těžký MLX byl tedy zamítnut z pohledu efektivity.

---

7. Outdated / Suboptimal Methods Ledger

V průběhu auditu byly na horkých trasách (hot paths) zachyceny metodické postupy nebo knihovny, jejichž přítomnost na limitovaném 8GB M1 prostředí lze klasifikovat jako překonanou a vysoce suboptimální. Tabulka přibližuje nutné substituce, logické zdůvodnění modernizace a realistický horizont provedení.

Aktuální metoda / Architektura	Moderní a špičková alternativa pro M1	Naléhavost a Realističnost náhrady
Standardní json nebo orjson	Přechod na strukturovaný balíček **msgspec** (tvorba dedikovaných msgspec.Struct místo volných dict objektů).[5]	**Kriticky naléhavá.** Výhoda leží v nulové redundanci a rapidně optimalizovaném uložení paměťových bloků v rámci C kontiguálních struktur. Realistické na provedení.
Naivní srovnávání vektorů scipy/numpy	Nativní hardwarová akcelerace instrukcí procesoru na úrovni křemíku – nasazení knihovny **SimSIMD** pro operace nad RAG vektory.[10]	**Vysoká.** Rychlé, s drop-in kompatibilním API a naprostou absencí overheadu, dramaticky urychlí vyhledávání podobností pro znalostní grafy, výkonový nárůst přesáhne 10x oproti starým

		přístupům.[11]
Hustá matice np.zeros(N,N) v GNN	Opustit pole nula-na-nula a přistoupit ke strukturám reprezentace přes Seznam Hran (Edge Index) nebo zapojit efektivní paměťový Rust back-end, knihovnu **rustworkx**.[1]	**Kritická a neodkladná.** Bez tohoto architektonického kroku se program při navýšení počtu uzlů autonomní znalostní báze plně zřítí v důsledku kolapsu dostupné unifikované paměti.
Těžké moduly pro metadata (PIL, pypdf, ffmpeg)	Prozkoumat možnosti asynchronního PyObjC mostu přímo k operačnímu systému macOS (frameworky **Vision, AVFoundation, PDFKit**).[1]	**Střední/Dlouhodobá vize.** Zabránit nárůstu procesů na pozadí, odstranit obrovské závislosti paměti, nutné hlubší studium programování nativních wrapperů pro Python, experiment vhodný pouze jako budoucí zkouška.
Hashing strings pomocí hashlib.sha256	Kde nejde o validaci integrity z důvodu bezpečnosti a hackingu, plošně nasadit knihovnu **xxhash** pro identifikátory entit a klíče do paměťové databáze.[1]	**Nízká/Střední naléhavost.** Okamžitý mikropřínos rychlosti (10násobné zrychlení na procesorovém taktu).[20]
Inline kompilace reg. výrazů (re.search(r"...", x))	Extrakce paternu na nejvyšší úroveň kódu modulu: uložení předpočítaných regulárních stavů jako **Precompiled re.Pattern** konstanty v hlavičce souborů.	**Vysoká.** Naprosto nulové náklady na úpravu kódu, obrovský a okamžitý přínos na snížení teploty procesoru v iterativním zkoumání.[1]

---

8. Top 20 Recommended Changes

Tento exekutivní seznam prioritních doporučení je seřazen na základě kombinace očekávaného dopadu na propustnost, miře přijatelného technického rizika na chodu celku a maximálního

shody se striktním modelem macOS + M1 8GB UMA limitace.

1. ** Záchrana paměti GNN:** V souboru gnn_predictor.py okamžitě odstranit sekvenci alokace np.zeros((n_nodes, n_nodes)). Přepsat graf do formátu seznamu hran (Edge Index 2xN) nebo využít pro běh grafu podklady enginu rustworkx.[1] *Proč nyní:* Zabraňuje garantovanému zamrznutí a pádům z důvodu "OOM". *Cílová metrika:* Snížení spotřeby RAM pro 45k uzlů z hrozivých 8 GB na jednotky MB.
2. ** Přidání LRU stárnutí do Graph Features:** V modulu gnn_predictor.py omezit absolutní kapacitu atributu self.node_features = {} prostřednictvím implementace fronty typové collections.deque(maxlen=...) nebo pevně ohraničeného mapovacího kontejneru s mechanismem LRU. *Proč nyní:* Ochrana proti plíživému a nenápadnému vyčerpávání VRAM (memory leak) u autonomního long-run běhu.[1]
3. **[HIGH] Přepis JSON serializační logiky na moderní Msgspec:** Uvnitř autonomous_orchestrator.py a úložného mechanismu evidence_log.py plně odstranit json knihovnu a nahradit pomalý a paměťově zničující postup pro parsování AnalysisSchema.from_dict zavedením architektury na bázi msgspec.Struct a msgspec.json.decode().[5] *Cílová metrika:* ~6násobně rychlejší dekódování do typů, a pokles režijní RAM zlomkových hodnot proti staré architektuře Pydanticu.
4. **[HIGH] Fixní integrace MLX GC Pipeline na konci tasku:** Provedení auditní kontroly napříč všemi hot-paths využívajícími MLX k vyhodnocování. Je naprosto nutné zavést standard pro materiálové dokončení grafu – sekvence logiky musí vždy striktně dodržet pořadí: nejprve výpočet materializovat příkazem mx.eval() a bezprostředně poté vyprázdnit VRAM paměťový zásobník zavoláním příkazu mx.metal.clear_cache().[1] *Cílová metrika:* Obnova až 20% čisté dostupné VRAM po uvolnění předchozí generace.
5. **[HIGH] Nasazení hardwarové vektorizace přes SimSIMD:** Uvnitř modulů provádějících kalkulaci kosinové podobnosti smazat importování modulu scipy.spatial a plně nasadit knihovnu SimSIMD. *Proč nyní:* Je to bezproblémové nahrazení funkcionality, jež přináší přesunutí drtivé matematiky na nezatížená výkonová křemíková jádra ARM. *Cílová metrika:* Zkrácení výpočetního okna RAG similarity o >90 % a >100x zrychlení na ARM.[10]
6. **[HIGH] Ošetření Lazy Import Blocks ve vlákně pro Background:** Okamžitě zajistit, aby veškera logika volající importlib.import_module na nejtěžší moduly (v souboru autonomous_orchestrator.py) byla provedena jako neblokující I/O úkol izolovaným asynchronním wrapprem, např. logikou pomocí asyncio.to_thread a nahradit synchronní OS systémová volání.[1] *Proč nyní:* Odstraní záškuby a pauzy blokující kritickou hlavní instanci smyčky event loopu.
7. **[HIGH] Vynucení MLX kv4 Cache a přehodnocení paměťových technik Draft modelu:** U enginu jazykového modelu hermes3_engine.py pevně aplikovat kvantizační techniku pro 4bit cache okno (KV4). Současné řešení speculačního dekódování je bez kvantizace rizikové pro RAM. Konfigurační detekce v init blocku nesmí být závislá na náhodné instanci, 4 bity musí být imperativ pro všechny modely v paměti na limitované 8GB architektuře.[17] *Cílová metrika:* Úspora více než gigabajtu na jedné alokované stopě modelu při větším RAG kontextu.[17]
8. ** Pre-kompilace regulárních formátů a dotazníků:** V těle analyzačních utilit

metadata_extractor.py, graph_rag.py a skórovacích technik fixně přesunout veškerá deklarovaná a inline psaná pravidla přesunem modifikátoru re.compile() z hlubokých funkcí bezpečně do sekce globálního konstantního načítání modulu.[1]

9. ** Radikální odstranění dvojité evidence a serializace JSON v cyklu:** Upravit kód spravující metodu záznamu událostí evidence_log.py a eliminovat podvrstvený zduplikovaný způsob vnitřního volání pro obgenerování hashe (uprostřed procesu append()), jež se nevědomě odkazuje a zbytečně opakovaně provolává zpět celostní vrstvu json.dumps() na totožná data.[1]
10. ** Asynchronní Thread Pool Limity a obříza kapacity workera:** Vymýtit kdekoliv uvnitř projektu dynamickou deklaraci vláken (typu využívajících hodnoty os.cpu_count() k alokaci limitu 8 procesů), například u I/O file fetching metod, a explicitně tak snížit maximální souběžnost striktně u všech instancí ThreadPoolExecutor nejvýše na číslo 2 či 3. Ponechat na 8GB stroji spuštěno maximálně osm procesních toků vede výhradně do stavu mrtvého zpomalení stránkováním sběrnice.[1]
11. ** Vytvoření odizolovaného CoreML ANE Embedder model kódu:** Experimentální pokus o export textového RAG embedding mechanismu ze závislosti na čipu GPU do kompilovaného nativního obalu vrstvy CoreML pro výpočetní matice tak, aby se úlohy počítaly mimo jakékoliv sběrnice pro běh LLM. Tento separátní tah přenese sílu na akcelerátor Apple Neural Engine (ANE), disponující fantastickou spotřebou a odděleným execution plánováním.[21]
12. ** Databázové zefektivnění a LMDB Batching Writes:** V souboru spravujícím vrstvu databázové stálosti a uložení knowledge/atomic_storage.py optimalizovat postupné po-jednom logování na efektivní a souhrnný způsob pomocí procedurálního zápisu put_many.[1]
13. ** Migrace z bezpečnostního SHA na rychlý Xxhash u stringů:** Analyzovat místa pro generování hash podpisů interních záznamů s nulovým bezpečnostním imperativem, a plošně pomocí vyhledávacího filtru na projekt modifikovat použití archaického hashlib.sha256 spuštěním bleskově se orientující nativní C knihovny xxhash.[1]
14. ** Kalibrace zpoždění diskových fsync operací z evidence logů:** Konfiguračně seřídit systém tak, aby četnost zápisu flush limitu bufferu stoupla ze stávajících zbytečně agresivních pauz na rozumný flush threshold po nasbírání například každých 10 tréninkových/evidence událostí.[1]
15. ** Ochrana ladících hlášení Logging Guardrails:** Přeuspořádání textových interpolací. Obalení rozsáhlých trasovacích zpráv spojených s drahým procesem textové nebo json kompilace do podmiňovacích výrazů s lenivou kontrolou stavu debuggeru (metodou if logger.isEnabledFor) ke všem hot-path voláním čítajícím tisíce operací.[1]
16. ** Odmítnutí optimalizace skórovačů mimo centrální P-jádro:** Zablokování nešťastného chování přesouvat O(1) počty a scoring funkce (typu Action Scorers) ze základního procesoru na grafické rozhraní, čímž by se platil gigantický příplatek paměťové kopie a overheadu v API komunikaci pro zanedbatelnou aritmetickou operaci.[1]
17. ** Nativní Metadata Extractor via OS C-framework:** Jako inovační dlouhodobý cíl prozkoumat zaházení Pythoních gigantických závislostí PIL/ffmpeg pro zpracování

metadat těžkých souborů obrazu a videí. Přes PyObjC most integrovat přímo vestavěné macOS rutiny (přes Vision a AVAsset systém), čímž dojde k ochránění striktní hranice 500 MB špičkové kapacity alokací na samotnou jednu extrakci metadat dokumentu.[1]

18. ** Optimalizace přetváření textu (Prefill) pro System Cache:** Zkontrolovat modelovou iniciaci v hermes3_engine.py. Odstranit zbytečné znovuvytvoření a opakovanou konkatenci masivního stringového základu s příkazy (tzv. system_prompt). Zajistit jednorázový předvýpočet a sázet tento prefill pouze načtením fixních databází .npz souborů uložených ve sdíleném balíčku formátu msgspec.[1]
19. ** Omezující parametry a stropy paměti pre Reranker komponentu:** Pro komponentu definovanou jako LightweightReranker hardwarově zafixovat omezený počet dokumentů určených ke zpětnému načítání a rankování, v zájmu nevybočení z mantinelů spotřeby na E-cores CPU systému.[1]
20. ** Refaktoring BudgetManager limitů a kontrola záchranných brzd:** Podrobit logiku fail-safe parametrů auditu a prověřit deklarovanou nesouladnost – zprávy referují 5.5 GB coby reálný limit na GPU, ale implementace sledovacích mechanismů v logice třídy BudgetManager přikazuje vypnutí orchestrátoru na poněkud nesmyslně nízkých agregovaných objemech kolem pouhých 2.0 GB. Je třeba upravit hodnotu k absolutní harmonizaci s OS detektorem fyzického využití VRAM stopy.[1]

---

9. Suggested Execution Order

Plán na implementaci nesmí narušit chod stávajících procesů. Fáze kopírují míru technologického rizika a vliv na efektivitu 8GB cílového stroje.

Phase 1: Safest wins (Kritické záplaty bez ohrožení architektury, provedení okamžitě)

- Odstranění zničující naivní a hrubé implementace pole np.zeros do neuronového grafu sídel v kódu modulu gnn_predictor.py (Okamžitá izolace před selháním běhu z důvodů udušení propustnosti OOM chybou RAM paměti). Nasazení adekvátní hranové architektury.
- Implementace hardwarové vektorizace nahrazením nativních procedur knihovnou SimSIMD pro úkony sémantické blízkosti.
- Nekompromisní dořešení správy odvozených instancí pro grafickou cache u MLX generátoru (vynucené párování uvolňovacích funkcí: spuštění evaluace mx.eval() vždy ihned obsaženě před fází samotného odklízecího povelu pro clear_cache()).
- Plošná a automatizovaná separace a prekompilace regulárních výrazů do hlavičkových hlídačů formátování a nahrazení drobných podpisů přechodem na knihovní základ funkce xxhash.

Phase 2: Measurement-first (Náročnější iterace a architektonické zásahy vyžadující profilování dat, horizont týdenní zkoušky)

- Komplexní logický refaktoring řídicího enginu i datových sběračů; náhrada zastaralé

filozofie ukládání s json nebo Pydanticem na nejdynamičtější C modul msgspec. Implementace vyvolá potřebu robustnější integrace ověřování platnosti přesných definic datového rozhraní (return byte types).
- Kalibrace limitů na paralelní chod workerů na úrovni I/O v souboru orchestrátoru, včetně nápravy k chybám náchylných procesů z "asynchronních blokací lazy load" sekvencí.
- Zavedení plné redukce a zapnutí komprimační kvantizace interního prostoru pro okno KV cache u paměti spekulačních generátorů textu. Prvně měřit, nakolik destrukce formátu cache na čtyři bity omezila kvalitu dedukce RAG a celkovou bystrost úsudku u obou současně držících se modelů na Apple hardwaru.[17]

Phase 3: Apple-native experiments (Inovace odemčené optimalizací HW rozhraní)

- Samostatný pilotní subprojekt zahrnující úplné odpojení menších procesů zátěže vektorového kódu textových embedderů ven ze softwarového spouštěcího grafu GPU/MLX a jejich transformaci na zkompilované modely CoreML platformy. Tak dojde k exkluzivnímu přiřazení výpočetní práce pro embeding natvrdo na obvody Neural Engine (ANE). Výsledkem bude markantní sražení teploty systému, téměř nulová konzumace ve wattech pro malý obvod a odstranění soutěžního zápasení na hlavní 8GB RAM větvi.

Phase 4: Only-if-needed architectural changes (Vratná rozhodnutí k nasazení při dlouhodobé neudržitelnosti)

- V případě neudržitelnosti spotřeby RAM pro stahování z webu, přejít k přemodelování modulu extrakce dat do formy nativních C-wrapperových obálek pro API most PyObjC a zkoušet neobvyklou přeměnu práce mimo obvod ffmpeg pro záchranu V/V výkonu celého ekosystému s pouhou osmičkou sdílené paměti.

---

10. Final verdict

Výsledek nezávislého auditu zdrojových kódů a provázání teoretických hypotéz s praxí u projektu Hledač Universal odhaluje naprosto obdivuhodnou počáteční technickou vizi. Tvůrci softwaru mimořádně silně respektovali extrémní mantinely stroje poháněného 8GB sdílené unifikované paměti UMA (což dokazují techniky jako precizně obalené Semafory a promyšlené guardrails zabraňující alokaci vícera těžkých LLaMA vrstev napříč systémem). Nicméně se v průběhu růstu projektu do kódu nepozorovaně vkradly zcela smrtící a plíživé hrozby limitů architektury Apple M1.

Skutečným ohrožením na cestě za maximální autonomní bystrostí systému nejsou aktuálně hranice výpočetní hrubé síly procesoru P-Core nebo maximální hodinový takt frekvence grafických Metal stínačů; kritickou bariérou je drcení propustnosti operační paměti a její nevyhnutelná fyzická fragmentace. Obrovskou daň si vybírají postupy O(N) dict struktur z Pythonu požírající L2 a L3 cache čipů, brutální přesouvání datových zátěží bez potřeby (jako u dvojité verify_integrity sériové konverze) a především skryté časované OOM bomby

maskované do zdánlivě banálních matematických obsluh typu np.zeros(N,N) v grafové prediktivní síti M1.

What should definitely be done:

1. Fix architektonického designu GNN struktur od alokací z $O(N^2)$ polí k bezpečné reprezentaci v paměťově nulovém hranovém profilu, případně nahrazení motorem rustworkx. Záchrana ~8 GB alokací.
2. Radikální vyhladovění veškerých JSON logik a parsingu na rychlý kompilovaný strukturální most přes framework msgspec.
3. Nahrazení pomalé standardní algebry v maticích za Apple-nativní a SIMD/AMX akcelerovanou vrstvu dot-produktů a kosinových srovnání skrze nasazení SimSIMD.[10]

What should only be benchmarked:

Aktivace a vnucení KV4 komprese mezipaměti jazykových LLM modelů během fungování metod pro spekulativní návrhy tokenů, spojené s vyhodnocováním kvality vypsaných kontextů a halucinování dlouhých OSINT textů. Posunování procesů blokujícího stahování a ukládání k logice do vláken E-Core přes omezená pole workerů.

What should be explicitly avoided on M1 8GB:

Absolutně doporučuji odmítnout navrhovanou tezi přesunout v rámci programu jakékoliv malé nebo dílčí jednoduché matematické numpy kalkulace k backendové grafice procesem zapojení modulu PyTorch MPS, kde vysoké čekací časy na zotavení linky Metal pro 8GB přehluší původní benefit akcelerace. Dále striktně vynechejte veškeré pokusy k paralelizaci a spouštění masových worker threadů na hodnoty vyšší než 3. V prostředí omezeného Apple čipu je čistota RAM pro 6-7 GB obsazených textovými váhami nedotknutelná. Následování výše uvedených principů zajistí, že Hledač Universal dosáhne teoretického propustného maxima a jeho agent udrží stabilní běh i u permanentní, nekonečné analýzy kyberprostoru bez pádů systému.

Citovaná díla

1. EXTREME_OPTIMIZATION_AUDIT_M1_8GB.md
2. Profiling Apple Silicon Performance for ML Training - arXiv, použito března 7, 2026, https://arxiv.org/pdf/2501.14925
3. Thoughts on Apple Silicon Performance for Local LLMs | by Andreas Kunar | Medium, použito března 7, 2026, https://medium.com/@andreask_75652/thoughts-on-apple-silicon-performance-for-local-llms-3ef0a50e08bd
4. A quick benchmark comparing msgspec (https://github.com/jcrist/msgspec), pydantic v1, and pydantic v2 · GitHub - gists · GitHub, použito března 7, 2026, https://gist.github.com/jcrist/d62f450594164d284fbea957fd48b743
5. Benchmark: msgspec vs. Pydantic v2 | Backend APIs, Web Apps, Bots & Automation, použito března 7, 2026,

https://hrekov.com/blog/msgspec-vs-pydantic-v2-benchmark
6. Benchmarks - msgspec, použito března 7, 2026, https://jcristharif.com/msgspec/benchmarks.html
7. My first ever article: "Finding the fastest Python JSON library on all Python versions (8 compared)" - Reddit, použito března 7, 2026, https://www.reddit.com/r/Python/comments/1ah4d2t/my_first_ever_article_finding_the_fastest_python/
8. Rustworkx Comparison Benchmarks With Other Libraries, použito března 7, 2026, https://www.rustworkx.org/benchmarks.html
9. [joss] Graph size limits · Issue #623 · Qiskit/rustworkx - GitHub, použito března 7, 2026, https://github.com/Qiskit/retworkx/issues/623
10. simsimd - PyPI, použito března 7, 2026, https://pypi.org/project/simsimd/5.1.0/
11. SimSIMD v2: 3-200x Faster Vector Similarity Functions than SciPy and NumPy - Reddit, použito března 7, 2026, https://www.reddit.com/r/Python/comments/170p7qm/simsimd_v2_3200x_faster_vector_similarity/
12. msgspec - PyPI, použito března 7, 2026, https://pypi.org/project/msgspec/
13. Benchmark of msgspec, orjson, pydantic, ... taken from Python discord - GitHub Gist, použito března 7, 2026, https://gist.github.com/jcrist/80b84817e9c53a63222bd905aa607b43
14. Drawbacks of Msgspec Compared to Pydantic: A Deep Dive with Examples - Hrekov, použito března 7, 2026, https://hrekov.com/blog/msgspec-vs-pydantic-drawbacks
15. What are the best libraries to work with graphs? : r/Python - Reddit, použito března 7, 2026, https://www.reddit.com/r/Python/comments/185xexg/what_are_the_best_libraries_to_work_with_graphs/
16. GitHub - ashvardanian/SimSIMD: Up to 200x Faster Dot Products & Similarity Metrics — for Python, Rust, C, JS, and Swift, supporting f64, f32, f16 real & complex, i8, and bit vectors using SIMD for both AVX2, AVX-512, NEON, SVE, & SVE2, použito března 7, 2026, https://github.com/ashvardanian/SimSIMD
17. UMA-Native KV-Cache Benchmarks on M4 Pro 64GB – kv4 Outperforms Unquantized · ml-explore mlx · Discussion #3134 - GitHub, použito března 7, 2026, https://github.com/ml-explore/mlx/discussions/3134
18. An Introduction to Speculative Decoding for Reducing Latency in AI Inference - NVidia, použito března 7, 2026, https://developer.nvidia.com/blog/an-introduction-to-speculative-decoding-for-reducing-latency-in-ai-inference/
19. Anyone doing speculative decoding with the new Qwen 3.5 models? Or, do we need to wait for the smaller models to be released to use as draft? : r/LocalLLaMA - Reddit, použito března 7, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1rgp2nu/anyone_doing_speculative_decoding_with_the_new/
20. SHA-256 vs xxHash | Compare Leading Cryptographic Hashing Algorithms - SSOJet, použito března 7, 2026,

https://ssojet.com/compare-hashing-algorithms/sha-256-vs-xxhash
21. Neural Engine outperforming the GPU on the M1 Max | by Steve Jones | Medium, použito března 7, 2026, https://blog.metamirror.io/neural-engine-outperforming-the-gpu-on-the-m1-max-243c7e780031
22. Benchmarking On-Device Machine Learning on Apple Silicon with MLX - arXiv, použito března 7, 2026, https://arxiv.org/html/2510.18921v1
23. Evaluating the Apple Silicon M-Series SoCs for HPC Performance and Efficiency - arXiv, použito března 7, 2026, https://arxiv.org/html/2502.05317v1
24. Using Accelerate and simd - WWDC 2018, použito března 7, 2026, https://nonstrict.eu/wwdcindex/wwdc2018/701/